

Statistical and Machine Learning

Neural Networks for Images

Instructions

All questions should first be discussed in pairs or small groups. Afterwards, we shall discuss them together. Solutions will be distributed after each session.

Question 1

From MLPs to CNNs. Multilayer perceptrons (MLPs) face three fundamental problems when applied directly to image data.

- (a) Briefly describe each of the three problems with using a plain MLP layer on image inputs $\mathbf{x} \in \mathbb{R}^{W \times H \times C}$.
- (b) What is *translation invariance*, and why is it desirable for image recognition? Illustrate with an example of what goes wrong without it.
- (c) How do Convolutional Neural Networks (CNNs) address these three problems?

Solution.

- (a) Three problems with applying a plain MLP to images:
 - 1. *Variable input size*: a different-sized weight matrix $\mathbf{W}$ would be needed for every image size, making the model inflexible.
 - 2. *Prohibitive number of parameters*: for a fixed image of size $W \times H \times C$ with D hidden units, the weight matrix has size $(W \times H \times C) \times D$, which is far too large for typical image dimensions.
 - 3. *Lack of translation invariance*: MLP weights are not shared across spatial locations, so a pattern learned in one position will not be recognised if the object is shifted to another position.
- (b) Translation invariance is the property that a model produces the same output regardless of where in the image an object appears. This is desirable because in many tasks we only care *whether* an object is present, not *where* it is. Without it, a weight vector tuned to detect a cross-shape on the left of an image gives a strong response (e.g. 5) there, but a much weaker response (e.g. 1) when the same cross is shifted to the right.
- (c) CNNs address all three problems via the convolution operation: small filters (e.g. 3×3) are slid over the image, so the same filter applies at any input size and drastically reduces the parameter count. Convoluting the filter at every location makes the model respond to a pattern wherever it appears, achieving translation invariance.

Question 2

Convolutions and cross-correlation. The 2D cross-correlation of a filter $\mathbf{W}$ of size $H \times W$ with an image $\mathbf{X}$ is defined as:

$$[\mathbf{W} \circledast \mathbf{X}](i, j) = \sum_{u=0}^{H-1} \sum_{v=0}^{W-1} w_{u,v} x_{i+u, j+v}$$

- (a) What is the difference between *convolution* and *cross-correlation*? Under what condition do the two operations coincide?

- (b) Compute the output $\mathbf{Y}$ of convolving the following 3×3 image $\mathbf{X}$ with the 2×2 filter $\mathbf{W}$ (using no padding):

$$\mathbf{W} = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \quad \mathbf{X} = \begin{pmatrix} 2 & 1 & 3 \\ 0 & 4 & 1 \\ 1 & 2 & 0 \end{pmatrix}$$

- (c) How does convolution implement *feature detection*, and what is a *feature map*?

Solution.

- (a) True convolution *flips* the filter before sliding it over the input; cross-correlation does not. In deep learning, the convention is to call the operation “convolution” even though cross-correlation is what is actually computed. The two operations coincide when the filter $\mathbf{W}$ is symmetric.
- (b) Since the input image $\mathbf{X}$ is 3×3 and the filter $\mathbf{W}$ is 2×2 , using no padding and stride 1, the output has size

$$(3 - 2 + 1) \times (3 - 2 + 1) = 2 \times 2.$$

We use the filter

$$\mathbf{W} = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

and slide it over all 2×2 patches of $\mathbf{X}$.

The four 2×2 patches are

$$\begin{pmatrix} 2 & 1 \\ 0 & 4 \end{pmatrix}, \quad \begin{pmatrix} 1 & 3 \\ 4 & 1 \end{pmatrix}, \quad \begin{pmatrix} 0 & 4 \\ 1 & 2 \end{pmatrix}, \quad \begin{pmatrix} 4 & 1 \\ 2 & 0 \end{pmatrix}.$$

Therefore,

$$Y_{1,1} = 1 \cdot 2 + 0 \cdot 1 + 0 \cdot 0 + (-1) \cdot 4 = 2 - 4 = -2,$$

$$Y_{1,2} = 1 \cdot 1 + 0 \cdot 3 + 0 \cdot 4 + (-1) \cdot 1 = 1 - 1 = 0,$$

$$Y_{2,1} = 1 \cdot 0 + 0 \cdot 4 + 0 \cdot 1 + (-1) \cdot 2 = 0 - 2 = -2,$$

$$Y_{2,2} = 1 \cdot 4 + 0 \cdot 1 + 0 \cdot 2 + (-1) \cdot 0 = 4 - 0 = 4.$$

Thus,

$$\mathbf{Y} = \begin{pmatrix} -2 & 0 \\ -2 & 4 \end{pmatrix}.$$

Equivalently, the convolution can be written as multiplication by a Toeplitz-like matrix. First vectorize the input image row by row:

$$\text{vec}(\mathbf{X}) = \begin{pmatrix} 2 \\ 1 \\ 3 \\ 0 \\ 4 \\ 1 \\ 1 \\ 2 \\ 0 \end{pmatrix}.$$

Each row of the Toeplitz-like matrix selects one 2×2 patch of the image and applies the filter weights. Since

$$\mathbf{W} = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix},$$

the Toeplitz-like matrix is

$$\mathbf{T} = \begin{pmatrix} 1 & 0 & 0 & 0 & -1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & -1 \end{pmatrix}.$$

Then

$$\text{vec}(\mathbf{Y}) = \mathbf{T} \text{vec}(\mathbf{X}) = \begin{pmatrix} 1 & 0 & 0 & 0 & -1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & -1 \end{pmatrix} \begin{pmatrix} 2 \\ 1 \\ 3 \\ 0 \\ 4 \\ 1 \\ 1 \\ 2 \\ 0 \end{pmatrix} = \begin{pmatrix} -2 \\ 0 \\ -2 \\ 4 \end{pmatrix}.$$

Reshaping this vector row by row gives

$$\mathbf{Y} = \begin{pmatrix} -2 & 0 \\ -2 & 4 \end{pmatrix}.$$

This uses the standard CNN convention, where “convolution” means cross-correlation, so the filter is not flipped.

- (c) The output at location (i, j) is large whenever the local image patch matches the filter $\mathbf{W}$, making the output “light up” wherever that feature is present. The output $\mathbf{Y} = \mathbf{W} \circledast \mathbf{X}$ is called a *feature map*: it represents the spatial distribution of a particular learned feature across the image.

Question 3

Padding, stride, and parameter efficiency. Consider a convolutional layer applied to an input of size $x_h \times x_w$, using a filter of size $f_h \times f_w$, with padding p_h, p_w on each side and stride s_h, s_w .

- (a) Write down the formula for the output size of the convolutional layer.

- (b) What is *valid convolution*? What is *same convolution*? What padding p (expressed in terms of filter size f) achieves same convolution?
- (c) How does the Toeplitz-like matrix structure of the CNN weight matrix reduce the number of parameters compared to a dense MLP layer? Briefly explain the roles of *sparsity* and *weight tying*.

Solution.

- (a) Output size:

$$\left\lfloor \frac{x_h + 2p_h - f_h + s_h}{s_h} \right\rfloor \times \left\lfloor \frac{x_w + 2p_w - f_w + s_w}{s_w} \right\rfloor$$

- (b) *Valid convolution*: no padding ($p = 0$); the filter never slides off the edge, giving a smaller output $(x_h - f_h + 1) \times (x_w - f_w + 1)$. *Same convolution*: zero-padding is added so the output matches the input size, requiring $p = (f - 1)/2$ on each side (exact when f is odd).
- (c) The CNN weight matrix is *sparse* (each output unit connects only to a small local patch, with all other entries zero) and *weight-tied* (the same filter weights repeat at every spatial location, i.e. a Toeplitz-like structure). Together these reduce the effective parameter count from $(W \times H \times C) \times D$ (MLP) to $f_h \times f_w \times C \times D$ (CNN), a massive reduction for typical image sizes.

Question 4

Pooling and normalization. Pooling layers and normalization layers are both important building blocks of CNNs, but serve different purposes.

- (a) Explain the difference between *max pooling* and *average pooling*. What property do pooling layers promote, and how does this differ from the *equivariance* property of convolutional layers?
- (b) What is *global average pooling*, and what practical advantage does it offer?
- (c) Batch normalization standardizes the activations within a minibatch $\mathcal{B}$. Write down the batch normalization transformation, define all terms, and explain the roles of the learnable parameters γ and β .
- (d) Why does batch normalization behave differently at *training time* versus *test time*, and how is this difference handled in practice?

Solution.

- (a) Max pooling takes the maximum activation over a local spatial window; average pooling takes the mean. Both promote *translation invariance*: the output responds similarly regardless of where within the receptive field the pattern occurs. This contrasts with the *equivariance* of convolutional layers, where a shift in the input produces a corresponding shift in the output (the feature location is preserved).
- (b) Global average pooling computes the spatial mean over all $H \times W$ locations of each feature channel, converting an $H \times W \times D$ feature map into a D -dimensional vector. Because the spatial dimensions are collapsed entirely, the classifier can accept images of *any size*.
- (c) Batch normalization replaces activation $\mathbf{z}_n$ with $\tilde{\mathbf{z}}_n$:

$$\hat{\mathbf{z}}_n = \frac{\mathbf{z}_n - \boldsymbol{\mu}_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}, \quad \tilde{\mathbf{z}}_n = \boldsymbol{\gamma} \odot \hat{\mathbf{z}}_n + \boldsymbol{\beta}$$

where $\boldsymbol{\mu}_{\mathcal{B}}$ and $\sigma_{\mathcal{B}}^2$ are the minibatch mean and variance, $\epsilon > 0$ is a small constant for numerical stability, and $\boldsymbol{\gamma}, \boldsymbol{\beta}$ are learnable per-layer scale and shift parameters that allow the network to undo the normalisation if needed.

- (d) During training, $\mu_{\mathcal{B}}$ and $\sigma_{\mathcal{B}}^2$ are recomputed each minibatch because the distribution of activations changes as parameters update. At test time, a single example may arrive, making batch statistics unreliable. The standard fix is to compute the population mean and variance over the full training set after training and freeze them alongside β, γ .

Question 5

CNN architectures. Several landmark CNN architectures have been developed for image classification, each introducing key innovations.

- (a) LeNet (1998) was an early CNN for digit recognition. What was its main application, and what accuracy did it achieve on MNIST compared to an MLP?
- (b) AlexNet (2012) marked a turning point in deep learning. List **three** architectural or training innovations that distinguished it from LeNet.
- (c) ResNet (2015) introduced *residual blocks* with skip connections. Write down the residual block update equation and explain, in words, why skip connections help train very deep networks.

Solution.

- (a) LeNet was developed by Yann LeCun for handwritten digit recognition (MNIST). It achieved 98.8% test accuracy after just 1 epoch, compared to 95.9% for an MLP, and was deployed by the US Postal Service.
- (b) Three innovations of AlexNet over LeNet (any three): (i) greater depth (8 trainable layers vs. 5); (ii) ReLU activations instead of tanh; (iii) dropout for regularisation; (iv) stacking multiple convolutional layers before pooling; (v) training across two GPUs. AlexNet reduced ImageNet top-5 error from 26% to 15%.
- (c) ResNet residual block:

$$\mathbf{x}_{l+1} = \varphi(\mathbf{x}_l + \mathcal{F}_l(\mathbf{x}_l))$$

The block $\mathcal{F}_l$ learns only the *residual* (the difference from the identity). The skip connection provides a direct gradient path to earlier layers, alleviating the vanishing gradient problem and enabling networks with hundreds of layers.

Question 6

Beyond image classification. CNNs can be applied to a variety of discriminative vision tasks beyond single-label classification.

- (a) Explain the difference between *image classification*, *image tagging*, and *object detection*. Why does image tagging replace the softmax output with independent logistic units?
- (b) Distinguish between *semantic segmentation* and *instance segmentation*. Give a concrete example illustrating the difference.
- (c) Describe the *encoder-decoder* architecture used for semantic segmentation. What information is lost in the encoder bottleneck, and how does the **U-Net** architecture address this?

Solution.

- (a) *Image classification* assigns a single label to the whole image (mutually exclusive outputs; softmax). *Image tagging* assigns multiple labels simultaneously; since tags are not mutually exclusive, a softmax is inappropriate and C independent logistic units are used instead, one per tag. *Object detection* returns bounding boxes and class labels for each individual object instance, combining localisation and classification.

- (b) *Semantic segmentation* assigns a class label to every pixel, grouping all pixels of the same class together without distinguishing separate objects. *Instance segmentation* predicts a class label and precise 2D mask for each individual object instance. Example: given two cars, semantic segmentation labels all car pixels as “car”, while instance segmentation separates them into “car 1” and “car 2”.
- (c) The *encoder* progressively reduces spatial resolution (via pooling or strided convolutions) to produce a compact high-level feature representation. The *decoder* upsamples back to full image resolution, yielding dense pixel-level predictions.

Question 7

Generating images by inverting CNNs. A CNN trained for classification can be “inverted” to generate images.

- (a) Explain how a discriminative classifier $p(y | \mathbf{x})$ can be converted into a (conditional) generative model $p(\mathbf{x} | y)$. Why is a prior $p(\mathbf{x})$ over images necessary?
- (b) The Langevin-style update for image generation is:

$$\mathbf{x}_{t+1} = \mathbf{x}_t + \epsilon_1 \frac{\partial \log p(\mathbf{x}_t)}{\partial \mathbf{x}_t} + \epsilon_2 \frac{\partial \log p(y = c | \mathbf{x}_t)}{\partial \mathbf{x}_t} + \mathcal{N}(\mathbf{0}, \epsilon_3^2 \mathbf{I})$$

Explain the role of each of the three terms (ϵ_1 , ϵ_2 , ϵ_3). What happens when $\epsilon_3 = 0$?

- (c) What is *neural style transfer*? Define the three components of its energy function $\mathcal{L}(\mathbf{x} | \mathbf{x}_s, \mathbf{x}_c)$ and explain what the *Gram matrix* captures and why it is used for the style loss.

Solution.

- (a) Define a joint distribution $p(\mathbf{x}, y) = p(\mathbf{x})p(y | \mathbf{x})$. Conditioning on a specific class $y = c$ gives the conditional generative model $p(\mathbf{x} | y = c) \propto p(\mathbf{x})p(y = c | \mathbf{x})$. A prior $p(\mathbf{x})$ is necessary because the discriminative classifier was trained to discard information not needed for classification, making $p(y | \mathbf{x})$ non-invertible. Without a prior, optimisation produces adversarial images that fool the classifier but are not visually realistic; the prior acts as a regulariser restricting generation to plausible images.
- (b) The three terms serve distinct roles: the ϵ_1 term (gradient of $\log p(\mathbf{x}_t)$) pulls $\mathbf{x}$ toward high-prior-probability regions, i.e. images plausible under the prior; the ϵ_2 term (gradient of $\log p(y = c | \mathbf{x}_t)$) pushes $\mathbf{x}$ toward images the classifier scores highly for class c ; the ϵ_3 term adds Gaussian noise for stochasticity and sample diversity. Setting $\epsilon_3 = 0$ removes the noise and gives a deterministic algorithm that approximately finds the single most likely image for the class.
- (c) Neural style transfer generates a new image $\mathbf{x}$ that re-renders a content image $\mathbf{x}_c$ in the visual style of a style image $\mathbf{x}_s$, by minimising:

$$\mathcal{L}(\mathbf{x} | \mathbf{x}_s, \mathbf{x}_c) = \lambda_{TV} \mathcal{L}_{TV}(\mathbf{x}) + \lambda_c \mathcal{L}_{\text{content}}(\mathbf{x}, \mathbf{x}_c) + \lambda_s \mathcal{L}_{\text{style}}(\mathbf{x}, \mathbf{x}_s)$$

The TV loss regularises the image to suppress high-frequency artefacts. The content loss matches CNN feature maps of $\mathbf{x}$ and $\mathbf{x}_c$ at a chosen layer, preserving high-level structure. The style loss matches *Gram matrices* between $\mathbf{x}$ and $\mathbf{x}_s$, where the Gram matrix $G_\ell(\mathbf{x})_{c,d} = \frac{1}{H_\ell W_\ell} \sum_{h,w} \phi_\ell(\mathbf{x})_{h,w,c} \phi_\ell(\mathbf{x})_{h,w,d}$ captures the co-occurrence statistics of feature activations across all spatial locations (the uncentred feature covariance). Matching Gram matrices reproduces the statistical texture of the style image without copying its spatial layout.